

AI 模型訓練

從資料到模型

把資料轉換成可以帶走的知識，成為一套面對未來的應對法則。



資料



模型



預測

Cocoya 【AI訓練積木】 參數教學

第 1 頁 · 封面

調參數 = 調整學習方式

積木上的每個數字，都是「**電腦怎麼學習這組資料**」的控制開關。

預設值就像一份**讀書計畫範本**——對多數人都不錯，**但沒有人的情況完全一樣**。

為什麼要調？

資料難易不同、圖片多寡不同、目標不同（迴歸？圖像分類？物件偵測？）。調整參數，讓「學習方式」配合你的資料。

接下來...

1. **基本設定**：類型、訓練主機、資料集、資料同步、模型存放
2. **餵資料的策略**：驗證比、擴增、Dropout
3. **電腦怎麼學**：模型 → Loss → 學習率(LR) → Epochs → Batch
4. **進階設定**：預訓練、微調、DNN 層、優化器、模型輸出

 **類型與訓練主機**：先決定「做什麼、在哪做」

類型 (TYPE)

- **分類器**：這張圖是什麼？（剪刀石頭布）
- **偵測器**：東西在哪裡？（框出物件位置）
- **循線**：線的方向（循跡小車用）
- **表格**：從表格資料學規律（非圖像）

 類型決定訓練方式與推論引擎，要與執行預測時「初始化模型」積木的類型一致。

訓練主機 (TRAINING HOST)

- **本地**：用自己的電腦訓練。資料少、模型小時最方便，預設。
- **遠端**：送到算力較強的遠端主機訓練。第一次執行會跳出 SSH 設定。

 **建議**：先用「本地」把整個流程跑通；等資料變多、訓練變慢，再考慮遠端。

資料集 DATASET

放哪裡？

```
專案目錄/dataset/classifier_dataset(資料集名稱)
/scissors 剪刀 138 張
/rock      石頭 108 張
/paper     布   124 張
/none     只有背景 32 張
```

每個子資料夾 = 一個分類，照片放在對應資料夾。

比喻：教科書

資料集就像「**章節清晰的教科書**」。
沒分章節、內容少，很難讀出重點。

⚠️ 每個分類**至少 5 張**，建議 **20-100 張**。
若數量不平均（30 vs 500）——這叫**類別不平衡 (class imbalance)**——樣本多的分類會主導Loss(之後說明)，訓練出來的模型容易偏心。

資料同步與模型存放

資料同步 (DATA SYNC)

僅在【訓練主機】 設為【遠端】時生效：決定本地資料集要不要送上遠端主機。

- **智慧**（預設）：資料有變更才上傳——最快、最省流量
- **每次**：每次訓練都完整上傳——最保險但最慢
- **略過**：不上傳，直接用遠端主機上既有的資料集

 訓練主機 = 本地時，此欄位沒有作用，保持預設即可。

模型存放 (MODEL FOLDER)

- 訓練好的模型會存到這個資料夾。
- **自動**從資料集名稱取出（dataset/xxx → model/xxx），**唯讀、不必手改**。
- 之後的推論積木「初始化模型」就是到這裡找模型檔。

 在 cocoya 專案根目錄下，有兩個指定的資料夾：
dataset 放資料集，model 放模型。

驗證比 VALIDATION SPLIT：留一份「模擬考」

 比喻：考題不可外洩

不能拿「讀過的題目」考自己——那只是背答案。
要留一部分不曾拿來讀的例題當模擬考，才知道「真的懂了沒」。

Validation Split (驗證比) = 從資料中抽出當作模擬考的比例（預設 20%）。

把原始資料集切成 train / validation 兩份資料集

Dataset Manager 拍好的一整包照片只有一份，訓練前由程式切開成兩份：

- train (80%)：作為課本 → 閱讀(訓練)後進行小考並更新權重（修正認知）
- validation (20%)：模擬考用，不可外洩。→ 課本讀完一次才進行模擬考一次，不更新權重（考題不進入認知學習）

兩份完全不重疊——所以模擬考才沒被「讀過」。

 實務：分層抽樣

「隨機抽 20% 當驗證」有缺點——若某分類數量很少，可能沒抽到，模擬考就沒考到它。

所以 Cocoya 用**分層抽樣 (Stratified Split)**：每個分類各自抽出固定比例，保證公平。

找到 4 個分類：

none: 32 張 / paper: 124 張 / rock: 108 張 / scissors: 138 張

分層抽樣 (stratified split):

none: train 26 / val 6

```
paper:    train 99 / val 25
rock:     train 86 / val 22
scissors: train 110 / val 28
(train = 讀課本+小考 / val = 模擬考)
```



本文件之後的比喻用法（統一用語）

比喻	指的是哪一份資料	終端機欄位
小考	用 train 訓練資料集 檢測——剛讀過的課本題， 拿來更新權重	accuracy / loss
模擬考	用 validation 驗證資料集 檢測——沒讀過的例題， 只評分、不更新權重	val_accuracy / val_loss

後面每一頁提到「小考」=在 train 上量；提到「模擬考」=在 validation 上量。⚠️ 只有 **模擬考** 能代表模型「真的學會了」；小考成績必然好看（讀過的題目），不能拿來判斷好壞。

💡 調校效果：變大（0.5）= 讀書資料變少、模擬考更嚴；變小（0.1）= 讀書多、但考得不夠。預設 0.2 平衡。

資料擴增 AUGMENTATION：把一份資料「舉一反三」

 比喻：變體練習

同一張圖，辨別時**轉一轉、翻一面、放大縮小**——看了同一題的「不同版本」。

這讓模型學會：「不管剪刀是橫的、斜的、近的、遠的，都是剪刀。」

Augmentation (擴增) 開啟後，每次餵進去的是**原始資料的輕微變體**，等於免費多了好多道練習題。

 調校效果

選項	後果
✓ 開啟 (預設)	資料多樣性↑ → 更強健、不易過擬合，多一點點訓練時間
✗ 關閉	只能重複看原圖，容易背圖；小資料特別容易過擬合

✓ 資料很少時**強烈建議開啟**；除非你刻意要原圖訓練，否則保持開啟。



隨機遺忘率 DROPOUT：強迫「換個思路想」



問題：死背 / 過度依賴

模型可能養成**依賴某條思考路徑**的壞習慣——記得 3 條規律就答對，但要是考的另 1 條就全錯。
這就像**只會一種解法**，考試題目換個樣子就卡住。



解法：DROPOUT

訓練時**隨機關閉一部分神經元**，逼它**不用原本那條路**、另找路徑解題。

比喻：把「代數」看成「幾何」，用**不同的思考方式**獨立得出答案 → 學會備份方案。



調校效果

數值	後果
0 (關閉)	沒有隨機遺忘，小資料易過擬合（全背起來）
0.2 (預設)	輕度約束，訓練時斷 2 成神經元 → 多數情況的好平衡
太大 (0.9)	神經元被關太多， 根本學不起來 （廢掉太多路徑）



怎麼觀察？打開訓練報告的曲線圖

(*_training_curve.png)：

· 訓練(train)準確率一直升、逼近 100%，但模擬考

(val)準確率停滯或下滑 → 兩條曲線張開成「剪刀差」 = 死背，可略增 Dropout

· **訓練 loss 持續下降、val loss 卻由降轉升** → 過擬合，同上

初學者保持 0.2 即可。

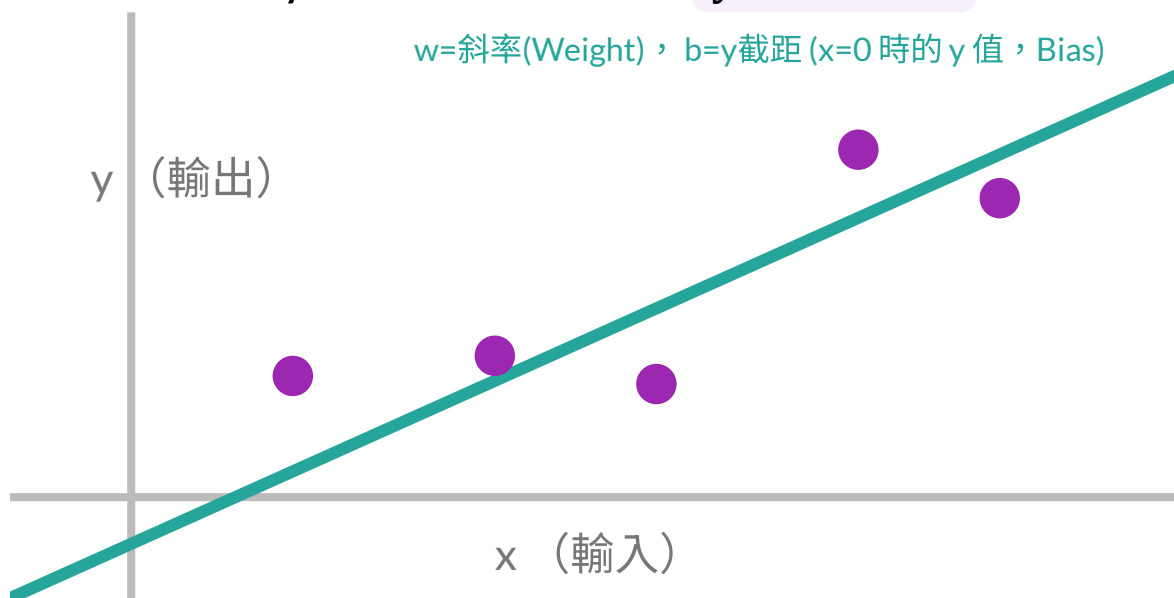
🌟 模型到底是什麼？

模型 = 網路架構(演算法) + 參數〔**權重 (Weight)** + **偏置 (Bias)**〕，一套數學應對法則。

訓練 = 從資料裡**找出這組參數**；之後拿它套用在新資料上做**預測 (Prediction)**。

📈 最簡單的例子：迴歸線

想用 x 預測 y ，可假設一條線 $\hat{y} = w \cdot x + b$



w (權重) 控制線的**斜率**， b (偏置) 控制線的**高度**。**找出好的 w 、 b = 訓練模型**。

💡 比喻：大腦的「應對法則」

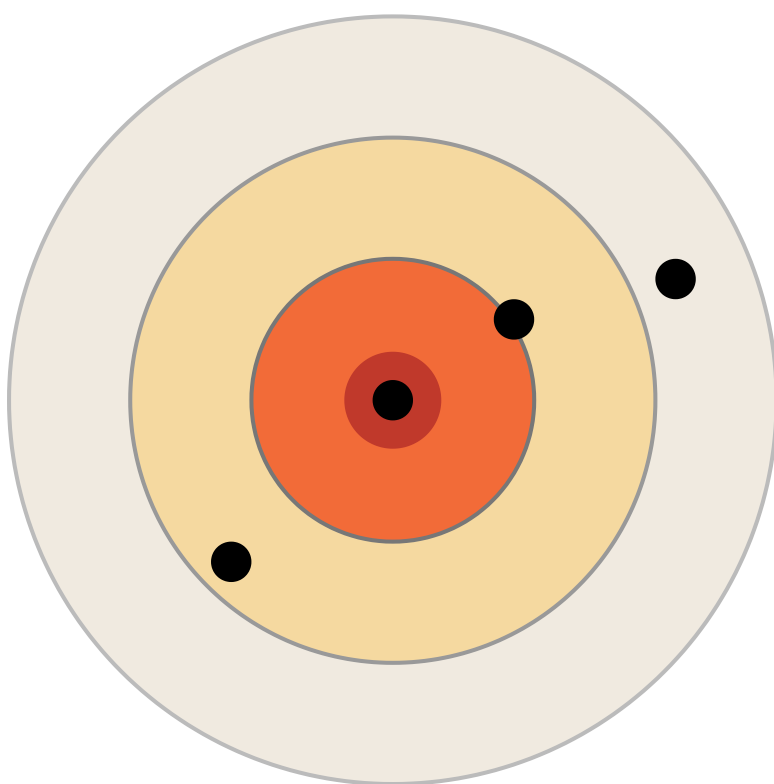
辨識「剪刀」不靠死背每一張圖，而是學會一組規則：「**兩根細長、交叉**」→ **剪刀**。

這個規則，就是用一組一組的 w 、 b **數值**儲存在模型裡——這就是可以帶走的知識。

💡 224×224 的圖將每個像素先正規化成 0~1 的數字後，輸入模型，經過層層運算，最後輸出「它是某分類的機率」。

 損失函數 LOSS FUNCTION：與答案「差多少」？

 預測有正中靶心(答案)嗎？



每支箭 = 一次預測；離紅心越遠，損失越大

 損失 = 預測與實際的差距

對每一筆輸入的資料，模型都猜一個答案。預測與答案**差多少**？把它**量化成分數**，就是**Loss (損失)**。

常用 **MSE** (Mean Squared Error, 均方誤差) 計算迴歸的損失：

$$MSE = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

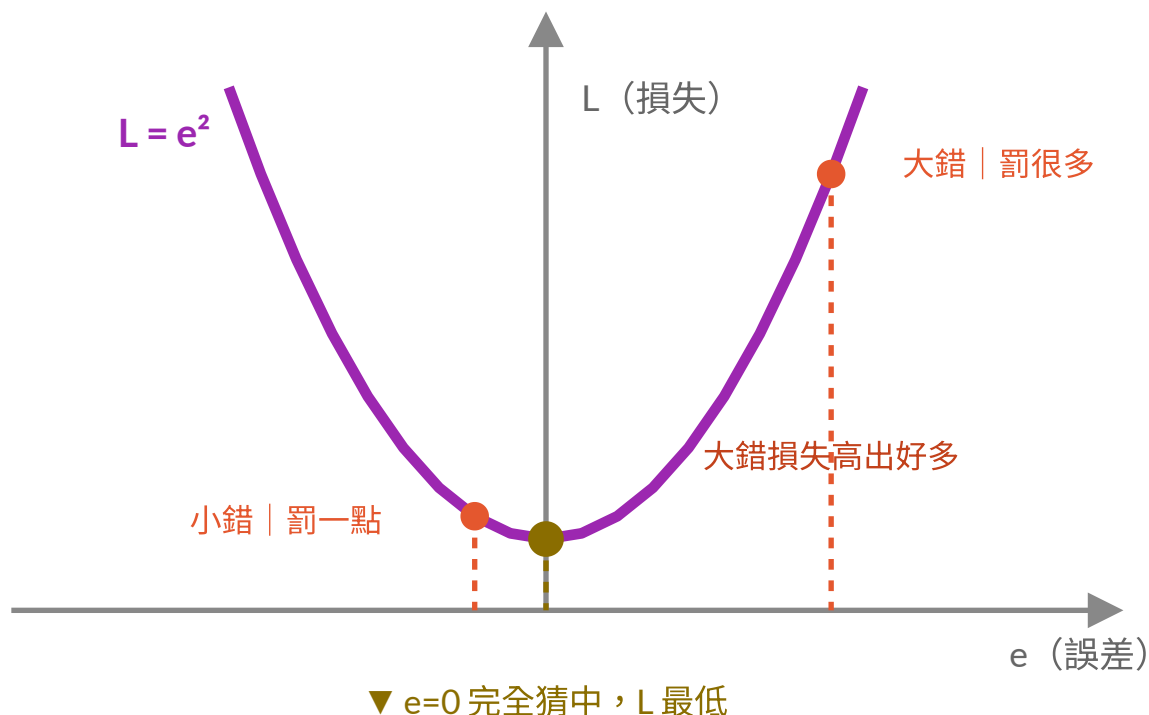
其中 $\hat{y}_i$ = 模型預測值、 y_i = 實際值、 n = 資料筆數。
也就是「平均(預測值 - 實際值)²」。

💡 **分類任務** (如剪刀石頭布) 的損失用「**交叉熵 (Cross-Entropy)**」，第 10 頁會用簡單的方式說明。

總之：Loss 越小 = 對目前資料猜得越準。

🔗 MSE 的二次曲線：偏離愈多、罰愈重

把 $(\hat{y} - y)$ 記為 e (誤差)，MSE 的每一筆就是 $L = e^2$ —— 一條開口向上的拋物線 (二次函數)。



🎯 從曲線看三件事

- **猜錯是平方級的罰**：差 1 罰 1、差 2 罰 4、差 4 罰 16——偏離越遠，代價暴漲。
- 左圖只是針對單一資料點，事實上我們要考慮 **所有資料的 L 加總平均**，所以曲線會有所變形。
- **谷底 = 曲線上 L 最小的位置**。真實資料有雜訊，通常**不可能每一筆都完全猜中** ($e = 0$) ——訓練要走到的是「**平均損失**」最低的谷底。

💡 訓練的終極目標：把所有資料的 L 加總平均，沿著這條拋物線 (其實是高維的「山谷」，目前先以線性迴歸模

型來想像。) 一路往最低點走，找到最貼近資料規律的 w 、 b 。

💡 因為 $e = (\hat{y} - y)$ ，而 $\hat{y} = wx + b$ ，調整 w, b 就可以調整 e ，進而改變 L 。訓練模型的過程，就是想辦法一步步調整 w, b 來讓 L 沿曲線降到谷底(最小值)。



交叉熵 CROSS-ENTROPY：分類任務的 LOSS，罰「錯得有自信」



分類任務怎麼給分？

分類任務中，模型不是猜一個數字，而是對每個分類給一個**機率**，合計為 100%。

例如輸入一張剪刀照片，模型可能說：

剪刀 72% 石頭 18% 布 10%
答案：剪刀 ✓

分類任務的 Loss 使用**交叉熵**的概念：只看「**正確答案那個分類**」的**機率**有多高，用 $L = -\log(p)$ 換算成分數：

- p 愈接近 1（猜對且有自信）→ L 愈接近 0
- p 愈接近 0（該猜對的沒把握）→ L 暴增到**非常大**



三種表現，罰多少？

模型的表現	答案的機率 p	交叉熵 $L = -\log(p)$
猜對，而且很有自信	0.9	很小（約 0.11）
猜對，但信心不足	0.4	中等（約 0.92）
猜錯，還錯得很有自信	0.05	非常大 （約 3.0）

（在電腦科學上，log 通常以 e 為底）

✓ 和 MSE 的精神相通：MSE 罰「偏得遠」，交叉熵罰「錯得有自信」。兩者都在教模型「對自己的答案要又準又有把握」。

💡 迴歸任務用 MSE，分類任務用交叉熵。
訓練曲線上的 loss，用的就是與任務對應的那一種。

 **學習率 LEARNING RATE**：依所在位置(斜率)向下修正時的**倍率**

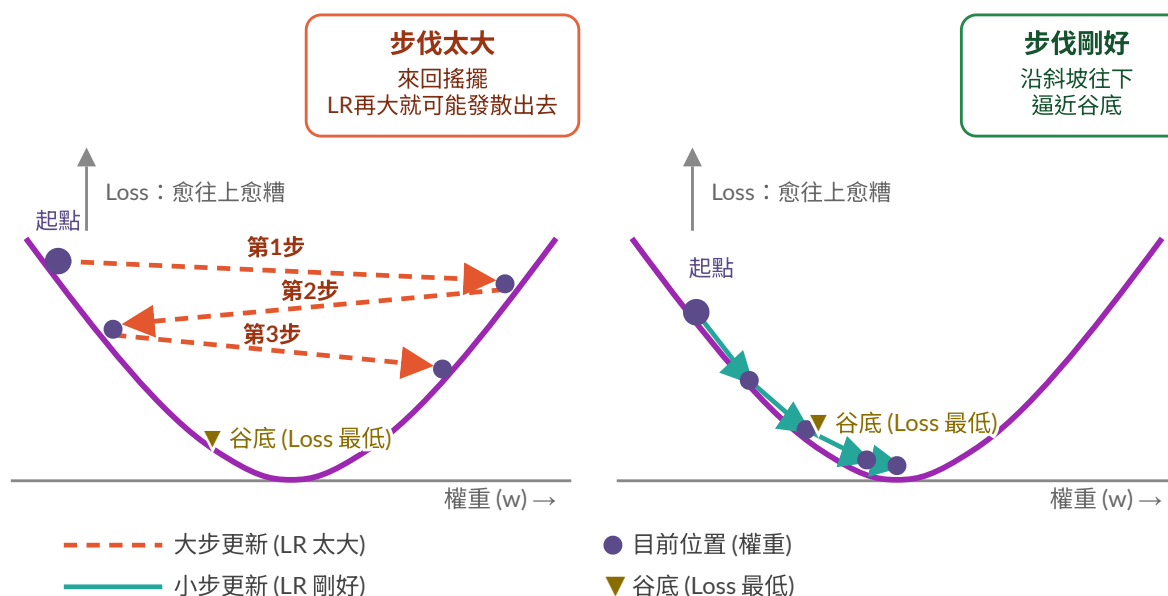
還記得大家要「把 Loss 愈推愈低」？做法是**沿著斜坡往下走**（梯度下降 Gradient Descent）。

Learning Rate (學習率, LR) 就是「**依斜率走一步的倍率**」——實際步長 = LR × 當下斜率：坡陡走大步、近谷底斜率小自動縮步。

學習率：太大 VS 剛好

LR 太大 → 搖擺、跨過谷底

LR 剛好 → 步步逼近谷底



調校效果

數值	下場	比喻
太大 (0.1~1)	一步跨過谷底，來回 搖擺不穩 ：Loss 曲線 震盪劇烈 ，甚至不降反升	修正太用力，錯一次改太多

剛好 (0.001)	每次穩定修正一點，收斂到低點：Loss 曲線 平滑穩定下降	看出小考錯在哪，重點修正
太小 (0.0001)	收斂極慢，很久才接近谷底：Loss 曲線 降得極慢、幾乎不動	每個字都讀、速度太慢

✅ 預設 0.001，不確定就先不動。

🔍 要調時，看 loss 曲線的走向決定：

- 曲線**震盪劇烈或不降反升** → LR 太大，調小
- 曲線**降得極慢、幾乎不動** → LR 太小，調大
- 曲線**平滑穩定下降** → LR 剛好，不用動

💡 一次只改 LR 一個參數，重跑再看曲線。

訓練輪數 EPOCH：整本書重讀幾遍？

定義

Epoch (輪) = 把整份訓練資料從頭到尾讀一遍。

Epochs = 讀幾遍。電腦每一輪，都把所有資料拿去更新權重一次。

讀太少 VS 讀太多

份數	後果	比喻
太少 (1~5)	還沒學到規律 (欠擬合)，Loss 還是很高	讀一遍就考試 → 全忘
剛好 (20)	學到規律，模擬考也有好表現	反覆複習到掌握
太多 (1000)	把資料「背」起來 (過擬合)，新資料反而變差	背好答案，考題一變就卡

 實際輸出：開始訓練 (20 epochs) → Epoch 1/20 → Epoch 2/20 ...每輪結束都會報一次 accuracy 與 loss。

 看「模擬考 (val) 分數」是否還在上升、還是停滯了？停了就代表多讀只是浪費時間。



BATCH SIZE 批次大小：一次「小考」考幾題？

EPOCH 與 BATCH 的關係

一整輪資料 = 一本課本。但電腦**不**一口氣讀完整本，而是拆成一小批一小批（Batch）。

每一輪 (Epoch) = 分批跑完整份資料

Batch 大小 = 每批處理幾張（預設 32）

每讀完一個 Batch，就做一次「小考」：算當下 accuracy、loss，並據此小幅更新權重。



調校效果

大小	下場
小 (1~16)	小考頻率高，修正快但有雜訊；每輪迴圈多 → 稍慢
中 (32) 預設	穩定與速度平衡
大 (128~512)	段考式，穩定但修正遲緩； 需更多記憶體

 記憶體不足（Out of Memory）就調小 Batch；想更穩就試著調大。



一次 Epoch 內部：1/11 → 2/11 → ... → 11/11（共 11 個 Batch），每一小段都是一次小考，結尾

顯示該輪總成績：

```
Epoch 1/5  
1/11 _____ 48s 5s/step - accuracy: 0.0938 -  
loss: 1.6940  
2/11 _____ 6s 686ms/step - accuracy: 0.1406 -  
loss: 1.6701  
11/11 _____ 13s 826ms/step - accuracy: 0.4829 -  
loss: 0.9899 - val_accuracy: 0.6667 - val_loss: 0.7169
```

看！accuracy 一路從 0.09 → 0.48 爬升，loss 一路下降——每批小考後都在進步。

預訓練 BACKBONE (骨幹網路)：站在巨人肩上

Backbone = 負責「看圖抓特徵」的網路骨架。它已**用上千萬張圖預先訓練**（預訓練，前備知識），懂得邊緣、形狀、紋理——你只要在它上面「補充」自己的類別。（積木欄位即顯示為「預訓練:」）
比喻：**站在巨人肩上**，不用從 0 發明怎麼「看」，只要微調它來認你的東西。

MOBILENETV2 預設

輕量快速，體積小、適合入門與 MCU。
→ 教學預設首選。

EFFICIENTNETB0

效率平衡，準度與運算量折衷。
→ 想更準一點的進階選擇。

RESNET50

深而重，特徵強但慢、耗記憶體。
→ 資料多、資源足時使用。

 圖片固定 224×224，是為了配合 Backbone 的預訓練輸入。換預訓練模型＝換一副「眼睛」，特徵取向不同。

 微調 FINE-TUNE：把「別人的筆記」改成自己的

 預設：骨架「凍」著

通常保持預訓練的「眼睛」不動，只訓練後面的思考層——又快又穩，資料少也安全。
這像「用別人整理好的筆記，只補自己的重點」。

 開啟 FINE-TUNE

解凍整個骨幹，也一齊微調它的權重 → 更貼合你的資料。
像「把整本筆記重寫成自己的話」。更準，但需要更多資料，否則把預訓練知識也搞壞。

 調校效果：

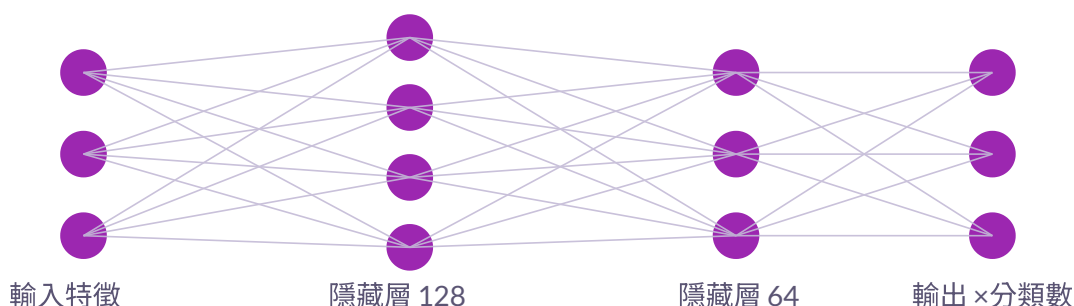
- 關閉（預設）：快、穩、資料少也 OK
- 開啟：準度可能更高，但易過擬合、訓練更久

 資料少（每類 < 20 張）建議維持預設關閉；資料多再嘗試開啟。

🧠 DNN 層 DNN LAYERS (自訂隱藏層)：多幾層「思考」？

在骨幹抓完特徵後，後面可以加幾層「思考層」（隱藏層 Hidden Layers）來做最終判斷。

格式是**逗號分隔的神經元數**，如 128,64（兩層）：
先 128 顆**廣泛想**，再縮到 64 顆**聚焦結論**。



輸入（骨幹特徵）→ 128 → 64 → 輸出（每個分類的機率）

每一顆神經元都**接到下一層的每一顆**，這種連法就叫「**全連接 Fully Connected**」——線越多代表組合方式越多，思考能力越強。

有幾個分類，輸出就有幾顆（Softmax 自動處理）
——輸入層與輸出層都不在積木中設定，只填中間的隱藏層即可。

💡 層越多越能表達複雜規律，但**越易過擬合、越慢、越吃記憶體**。留空 = 跳過思考層直接輸出。初學者用預設 128,64 即可。

⚙️ 優化器 OPTIMIZER (優化器)：讀書的策略

🔄 它做什麼？

決定「**這次該修多少**」——怎麼把權重從舊值一步步推到更低的 Loss。

基本公式（以 SGD 為例）：

$$\text{新權重} = \text{舊權重} - \text{學習率} \times \text{當下斜率 (Gradient)}$$

注意：**固定的是學習率**（就是你填的 lr 欄位，全程不變）。步長 = 學習率 × 當下斜率；斜率大步伐大（陡坡快跑），靠近谷底斜率小步伐自動縮小（精細修正）。

那 Adam 到底調什麼？它**不動你的 lr**，改的是「斜率」那一端——把斜率換成**自適應的步幅**。

📊 三種策略

優化器	特性
Adam 預設	記憶過去幾次修正的 方向與幅度 ，讓每個權重自動獲得適合自己的步幅 （不只調快、也會調慢）→ 前期收斂快、對初學者友善
SGD	固定學習率，步長全靠當下斜率 → 穩健踏實，但收斂較慢、易卡
RMSprop	對每個權重 個別調整 步伐（除以歷史根均方）→ 對不齊的梯度較友善

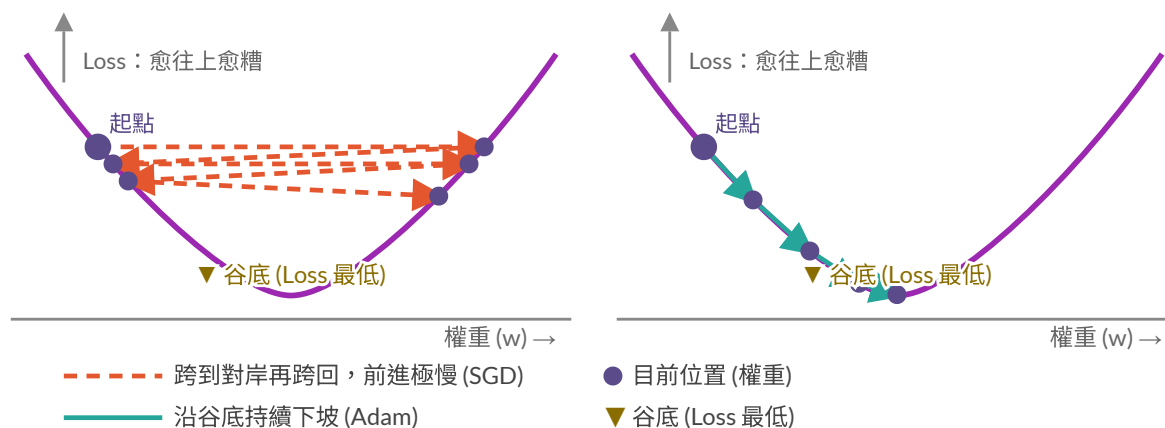
✅ 初學者用預設 Adam，幾乎不用改。



ADAM 真正解決的難題：狹窄谷地

× 沒有記憶 (SGD)：原地打轉

✓ 記住方向 (Adam)：持續下坡



谷地又**深**又**窄**時，SGD 只看「當下斜率」：一步往左、一步往右，**梯度互相抵消、原地磨**。Adam 記住前幾步的**方向**，左右搖擺平均掉之後**仍持續往谷底走**——這才是它「收斂快」的主因，不只是因為把步子放大。

另外，Adam 對「梯度長期偏小」的權重自動**放大**步幅、對「梯度大」的**縮小**步幅 → 讓每個權重的步幅量級一致，而不是一視同仁地亂快亂慢。



模型輸出格式 MODEL OUTPUT：最後交出什麼成品？

訓練結果要「包裝」成什麼形式儲存。預設 **none** = 只觀察成績、先不產模型檔（省下轉換時間）。

選項	成品	用在哪
none 預設	只跑訓練、看 loss/accuracy	調參數階段，避免浪費時間
int8	TFLite 量化模型（壓縮、省記憶體）	MCU / 邊緣裝置
f32	Float32 模型（精準）	PC 端 Python 推論
keras	原始 Keras 模型檔	想再繼續訓練/修改
int8+f32 / all	多種格式一次產出	同時要 PC 與 MCU 用

✓ 流程建議：先用 none 調好參數 → 確定 OK 再選格式正式產出模型。



訓練實戰：讀懂終端機訊息

把整條流程串起來：**掃描資料** → **分層抽樣** → **平衡類別** → **迭代訓練**。看到這些訊息，你就能知道電腦在做什麼。

```
找到 4 個分類: ['none', 'paper', 'rock', 'scissors']
  none: 32 張      paper: 124 張      rock: 108 張      scissors:
138 張
分層抽樣 (stratified split):
  none:      train 26 / val 6      ← 每類各自按 20% 抽模擬考，公平
  paper:     train 99 / val 25
  rock:      train 86 / val 22
  scissors:  train 110 / val 28

計算 Class Weight (平衡資料不平衡) ...
Class Weights: {0: 3.09, 1: 0.81, 2: 0.93, 3: 0.73} ← none最少→
權重最高，補救偏心

開始訓練 (5 epochs)...
Epoch 1/5
  1/11 _____ 48s 5s/step - accuracy: 0.0938 - loss:
1.6940
  2/11 _____ 6s 686ms/step - accuracy: 0.1406 - loss:
1.6701
 11/11 _____ 13s 826ms/step - accuracy: 0.4829 -
loss: 0.9899
- val_accuracy: 0.6667 - val_loss: 0.7169
```

💡 判讀重點：loss 降、accuracy 升 = 在學習；val (模擬考) 掉隊或暴漲 = 過擬合信號。



什麼情況，調什麼？診斷速查表

先看症狀，再動手。一次只改一個參數，改完再看結果。

你看到的症狀	最可能要調	方向
連訓練資料都猜很差 (欠擬合)	Epoch 太少 / 模型太小	增加 Epoch、加 DNN 層
訓練好、但模擬考很差 (過擬合)	Dropout / Augmentation / 資料	啟用擴增、增 Dropout、減少 Epoch
Loss 搖擺不穩、不降反升	Learning Rate 太大	調小 LR (如 0.001→0.0001)
收斂太慢、跑很久	Learning Rate 太小 / Epoch 多	調大 LR、減少 Epoch
記憶體不足 (OOM)	Batch Size 太大	調小 Batch (32→16)
類別數目不平均 (某類很少)	— (Cocoya 自動平衡)	確認 Class Weight 有印出；補更多照片
想更準且資料充足	Backbone / Fine-tune	換 EfficientNetB0 或開啟 Fine-tune

✓ 不確定時：**先用預設調好資料量**，資料品質才是決定準確度的最大關鍵。



小測驗（一）點選你的答案

Q1. 訓練時 Loss 一直搖擺、不降反升，最可能是哪個參數太大？

增加 Epoch

調大 Learning Rate

增大 Batch

開啟 Fine-tune

Q2. 訓練成績很好、模擬考（val）也很好，但拿到「新資料」卻變糟——這是？

欠擬合

過擬合

Learning Rate 太小

資料太乾淨



小測驗（二）

Q3. 每個 Batch 結束後，電腦會做什麼？

結束訓練

算一次 accuracy/loss 並小幅更新權重

重新掃描資料集

產出模型檔

Q4. 「分層抽樣」最重要的目的是？

讓訓練跑更快

每個分類都有公平的模擬考樣本

壓縮圖片

調高 Learning Rate

❏ 總結：記住這組口訣



一句話記住每個參數

- **資料集**：課本要整理好
- **模型** = 權重 + 偏置的應對法則
- **Loss**：離正確答案多遠
- **Learning Rate**：每步修正的幅度
- **驗證比**：留 20% 模擬考（分層抽樣）
- **Epochs**：整本讀幾遍（20）
- **Batch**：一次小考幾題（32）
- **Dropout**：強迫換思路，防死背（0.2）
- **擴增**：對原資料舉一反三（開）
- **預訓練**：站在巨人肩上（MobileNetV2）
- **優化器**：讀書策略（Adam）
- **DNN 層**：要多幾層思考（128,64）
- **微調**：把別人的筆記改成自己的（關）
- **輸出格式**：要交什麼成品（none 先看）



給你的工作流程

1. 先確定**資料品質**（最多的關鍵！）
2. 用「**none**」輸出，先跑預設看成績
3. 看 symptoms：欠擬合？過擬合？不穩？慢？**一次改一個**
4. 一切 OK → 再選 **int8 / f32** 正式產出模型

✓ 口訣：「資料要好、步幅要恰（LR）、遍數適中（Epoch）、防死背（Dropout/擴增）、一次只改一個。」

💡 下一步可以學：讀 *_training_curve.png 的 Loss 曲線，練習判讀「該停了沒」。

第 25 頁 · 總結

Speaker notes